

Eksperyment 4.5, Faza 4 – Hybrydowy override zasilany ciągłą obserwacją

Ten sam mechanizm twardego override z Eksperymentu 4.1 (0% → 90-97% detekcji flag), zasilony NIE krótkim, jednorazowym oknem, tylko ciągłą, godzinną obserwacją z Fazy 1 tego eksperymentu. Analiza w 100% na już zebranych danych – zero nowych, płatnych wywołań API.

1. PYTANIE I KONTEKST

DLACZEGO TO WAŻNE

Miękkie podpowiedzi Qdrant (Eksperyment 4.4, Faza 2-3 tego eksperymentu) konsekwentnie **szkodziły** detekcji flag bitowych u wszystkich 4 modeli LLM – nawet gdy sama biblioteka miała 93.5% trafności. Twardy override z Eksperymentu 4.1 działał świetnie, ale nigdy nie był testowany z danymi z ciągłej obserwacji. **Pytanie:** czy ten sam, niezmienny mechanizm, zasilony dłuższą obserwacją, też skorzysta – tak jak sugerowała cała hipoteza Eksperymentu 4.5?

2. DWA WARIANTY ŹRÓDŁA KLASYFIKATORA – NA TYCH SAMYCH 400 ODPOWIEDZIACH LLM

WARIANT A – TRIAL-WINDOW

Jak w oryginalnym mechanizmie
(Eksperyment 4.1)

47.2%

precyzja override'u przy klasyfikatorze liczonym na nowo z 30 ramek tej jednej próby – za mało danych, żeby heurystyka była wiarygodna.

WARIANT B – FAZA 1 (Ciągła obserwacja)

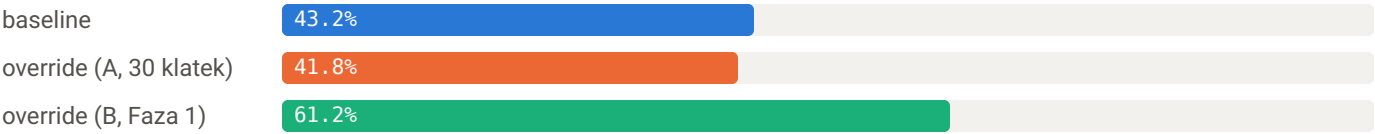
Werdykt z godziny/1,6mln ramek

100.0%

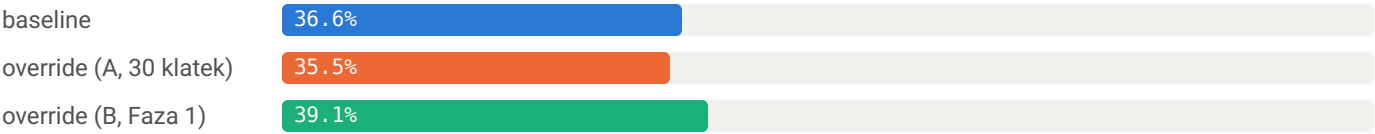
precyzja override'u (49/49 trafień, 0 fałszywych alarmów) – dokładnie ta sama właściwość, którą Faza 1 zmierzyła dla samego klasyfikatora.

3. WYNIKI – OGÓLNA DETEKCJA PER MODEL

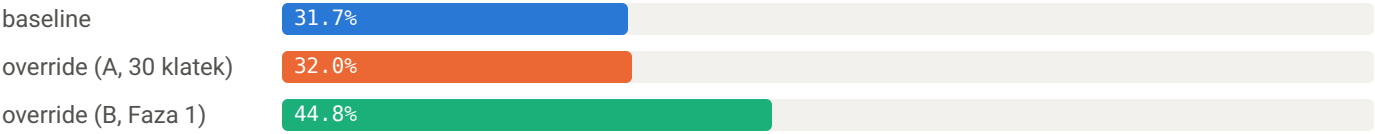
Claude



GPT



DeepSeek



Gemini



■ baseline (bez interwencji) ■ override A (30 klatek) ■ override B (Faza 1, 1h)

4. BIT_FLAG – KATEGORIA, DLA KTÓREJ TO WSZYSTKO ROBIMY

MODEL	BASLINE	OVERRIDE A	OVERRIDE B	Δ B VS BASELINE
Claude	25.4%	27.0%	53.1%	+27.7pp
GPT	29.1%	31.5%	30.4%	+1.3pp
DeepSeek	11.5%	26.4%	43.9%	+32.4pp
Gemini	28.8%	35.8%	52.2%	+23.4pp

KATEGORIA (AGREGAT 4 MODELI, OVERRIDE B)	BASLINE	OVERRIDE B	Δ
scalar (n=508)	75.2%	74.6%	-0.6pp (szum)
partial_scalar (n=52)	73.1%	73.1%	0.0pp
bit_flag (n=904)	16.2%	37.1%	+20.9pp

Interwencja jest chirurgicznie precyzyjna: poprawia wyłącznie bit_flag, nie rusza (w granicach szumu) pozostałych kategorii – spójne ze zmierzoną 100% precyzją samego override'u.

5. WNIOSKI I MOJE SPOSTRZEŻENIA (CLAUDE)

JAK JA TO WIDZĘ

To pierwszy wynik całej serii 4.1–4.5 o charakterze rozwiązania, nie tylko diagnozy. Poprzednie eksperymenty (4.3 Etap B, 4.4, Faza 2-3 tego eksperymentu) pokazywały GDZIE i DLACZEGO coś nie działa. Ten wynik pokazuje CO DZIAŁA – i to na tych samych 400 odpowiedziach LLM co wcześniej, zmieniając wyłącznie jedną zmienną (źródło danych klasyfikatora).

Dlaczego to działa, skoro miękkie podpowiedzi nie działały: override to podmiana struktury PO FAKCIE – model nie ma szansy jej zignorować, bo decyzja zapada poza nim. Problem z miękkimi podpowiedziami nigdy nie leżał w jakości biblioteki (Faza 2 ją naprawiła – 93.5% zamiast ~70%) – leżał w samym mechanizmie sugestii. Override obchodzi ten problem całkowicie.

Nierówny efekt u GPT (+2.5pp vs +13 do +18pp u pozostałych) – hipoteza: GPT rzadziej niż inne modele proponuje "czysty pojedynczy skalar" dla bajtów-flag, więc override ma mniej okazji, żeby w ogóle zadziałać. Niepotwierdzone, wymaga osobnej analizy.

Sugestia na następny krok: przenieść ten mechanizm do prawdziwego kodu C++ (DecodingAccuracyRunner::applyBitFlagOverride) – zasilić go długoterminową pamięcią per-pojazd zamiast krótkiego bufora RAM. To nie kolejny eksperyment, tylko uzasadniona tym wynikiem inżynieria.

6. OGRANICZENIA

- Override zadziałał tylko 49 razy na 400 prób (~12%) – dotyczy ograniczonego podzbioru sytuacji (propozycje "czysty pojedynczy skalar").
- Jeden seed (999) dla Fazy 1 i Fazy 3 – nie testowano generalizacji do innego rozkładu.
- Faza 1 i Faza 3 to dwa oddzielne uruchomienia generatora (ten sam schemat, inny przebieg w czasie) – statystycznie równoważne, nie identyczne klatka-po-klatce; w praktyce dokładnie odpowiada realnemu scenariuszowi (pojazd obserwowany godzinę, potem nowe zapytanie).

MagistrałaCAN4 – `esp_experiment_4_5_rpi/apply_override_offline.py` – wygenerowano automatycznie na podstawie wyników bieżącej sesji